

CSCE 701 CYBERSECURITY · MODULE 09 · LECTURE NOTES

Detection

How you would know.

MODULE OUTCOMES

- Design logging that answers security questions.
- Explain the base rate problem and its consequence for alerting.
- Build detections for specific threats and test them.
- Explain what to centralise and what to retain.
- Explain why most detection programmes fail.

1 Logging

LOG WHAT YOU WOULD NEED TO ANSWER THE QUESTION, NOT WHAT IS CONVENIENT TO EMIT

Most logging is written forwards — emit whatever the code happens to know at that point in execution — and is then largely useless during an investigation, because the field that matters was in a different function.

So design backwards: write down the questions you will need to answer, and then derive the fields required to answer them. 'Which accounts did this source address touch in the last week?' requires both the address and the account in every single event, which forwards-designed logging rarely achieves.

The security-relevant event types are a short list: authentication attempts, authorisation denials (Module 03 §3), privilege and role changes, configuration changes, secret access (Module 07), and administrative actions. Six categories, and getting those right beats logging everything.

And every event needs who, what, when, from where, and the outcome — the outcome field is the one most often missing, and a log of attempts without results cannot distinguish a successful attack from a failed one, which is the central question in every investigation.

EVERY SECURITY EVENT:

who the authenticated principal -- and the REAL one, not the service account acting on their behalf
what the action, and the specific object
when a synchronised timestamp, with timezone
where source address, and the request identifier
outcome permitted, denied, or errored

AND A CORRELATION IDENTIFIER threaded through every service, so that one user action is one searchable trace across the whole system (CSCE 678 Module 09).

WHAT NOT TO LOG

secrets, tokens, and passwords -- including inside error messages and request dumps, which is how they usually get there (Module 07 section 4)
personal data beyond what is needed, because logs are copied to aggregators, read widely, and retained for years
and whole request bodies, for the same reason

The 'who' field must be the real principal. A log showing a service account as the actor for every action is unusable for an insider investigation — which is Module 01 §2's adversary model arriving as a concrete logging requirement, and it is a common architectural mistake in systems where a middle tier acts on users' behalf.

2 Detection

- **Authorisation denials in bursts** (Module 03 §3) — **one principal generating many refusals is among the clearest signals available**, it is free to collect, and legitimate use produces very few of them.
- **Authentication failures across many accounts from a single source** — credential stuffing (Module 02 §4), **and it has a distinctive shape**: many usernames, one or few passwords, one origin.
- **A successful login from a new location immediately following a run of failures**, which is the signal that the stuffing *worked* — and is far more actionable than the failures alone.
- **New outbound destinations** (Module 04 §2) — **exfiltration and command-and-control both need a channel**, and a host contacting an address it has never contacted before is worth a look.
- **And privilege grants, secret reads, and configuration changes** — **low volume, high value, and they should be reviewed rather than merely stored. Low-volume high-value events are the best detections available**, precisely because a person can actually read them — and **most programmes over-invest in high-volume signals instead**, for reasons §3 explains.

AN UNTESTED DETECTION IS A HOPE

A detection rule that has never fired may be working perfectly in a quiet environment, or may be completely broken — and the two states look identical from the outside.

So trigger each rule deliberately, on your own system, and confirm the alert arrives — including that it reaches a person who knows what to do with it, which is the part usually left untested.

And re-test after changes, because a log format change, a field rename, a library upgrade, or a new service version silently breaks parsing — and the rule then matches nothing, forever, while continuing to appear enabled.

Which makes detection testing a continuous requirement rather than a one-off exercise — and the mature version is automated: synthetic attack traffic generated on a schedule, with the arrival of the alert as the test assertion. Which is CSCE 606's testing discipline applied to security controls, and it is this course's single most valuable operational practice.

3 The base rate problem

A 99%-ACCURATE DETECTOR ON RARE EVENTS PRODUCES MOSTLY FALSE POSITIVES

Ten million events a day, one genuine attack among them, and a detector with a 1% false positive rate. That is one hundred thousand false alerts and one true one.

So the precision is one in 100,001, and no team on earth can review that — which is CSCE 633 Module 12 §2's base rate problem, arriving in the setting where it does the most practical damage.

And the failure mode is entirely predictable: the alerts are ignored, then filtered, then muted, then deleted — so the detector's effective sensitivity becomes zero while the dashboard continues to show it enabled, which is worse than not having deployed it because it occupies the budget.

So the response is to raise the base rate rather than the accuracy: alert on rare, high-value events (§2), correlate several weak signals into one strong one, and scope rules narrowly to specific assets — and compute the expected alert volume before deploying any rule, which takes an hour and prevents the whole failure.

- Compute the expected daily volume before enabling a rule. If it exceeds what a person can actually review, the rule is not ready — and enabling it anyway is worse than not having it, because it degrades every other alert's credibility.
- Separate 'alert' from 'record'. Most signals should be searchable during an investigation and should not page anybody — and conflating the two is the commonest cause of excessive volume.
- Correlate. Three weak signals on the same principal within an hour is a far better detection than any one of them individually, and the base rate improves multiplicatively while the sensitivity falls only slightly.
- And measure the false positive rate per rule, then retire the rules that have never produced a true positive — which is a one-line query against your alert history and an uncomfortable conversation.
- Which almost nobody does. Rules accumulate and are never removed, which is why a mature detection stack is routinely noisier than a new one — and retiring rules is the maintenance nobody performs.

4 Why programmes fail, and what works

MOST DETECTION PROGRAMMES FAIL FOR OPERATIONAL REASONS, NOT TECHNICAL ONES

Alert fatigue (§3) — the volume exceeds the review capacity, so nothing is read, and this is the single largest cause.

Untested rules (§2) — a log parsing change silently disabled half the detections eighteen months ago and nobody noticed, because a detection that does not fire looks exactly like a quiet environment.

No response path. An alert that reaches someone who does not know what to do with it is not a detection — it is a notification — which is Module 10's subject and is the half of the problem that is not technical at all.

And logging the wrong things (§1) — the investigation needs a field that nobody emitted, which is discovered during the incident and cannot be fixed retroactively. That last one is why §1 designs backwards from the questions.

- **Start from the threat model (Module 01 §2)** — three specific threats, logged and detected properly and tested, beats thirty rules taken from a vendor's default list, because the three will fire meaningfully and the thirty will produce noise.
- **Log the short list (§1's six categories) with the five fields and a correlation identifier.**
- **Centralise, so that logs survive the compromise of the host that produced them** — clearing local logs is frequently an attacker's first action, and a log that can be deleted by the compromised machine is not evidence.
- **Retain long enough to investigate.** Intrusions are frequently discovered months after the initial access, and thirty days of retention answers nothing at all — which makes retention length a direct determinant of whether an investigation can succeed.
- **And test every detection, on a schedule (§2).** Centralisation and retention are the two unglamorous controls that decide whether an investigation is possible at all — and both are budget decisions rather than technical ones, which is why they are the ones to argue for first.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Google — Building Secure and Reliable Systems, the detection chapters (free PDF)	§1 and §4 from operators — including the logging-design-backwards argument.
Axelsson — The Base-Rate Fallacy and Intrusion Detection (free)	§3, with the arithmetic — the paper that should have settled how alerting is designed and did not.
MITRE ATT&CK (free)	A catalogue of adversary techniques with detection guidance per technique — useful for §2's rule design, and best used threat-model-first rather than as a checklist.

Resource	What it covers for this module
Atomic Red Team (free)	§2's detection testing, automated — small, safe tests that trigger specific detections, for use on your own systems.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Write five questions** you would need to answer during an incident, then check whether your logs can answer them.
- 2 **Add the missing fields** and re-check.
- 3 **Verify your logs record the real principal** rather than a service account.
- 4 **Grep your logs for secrets** and fix whatever you find (Module 07 §4).
- 5 **Build a detection for authorisation denial bursts** and trigger it on your own system.
- 6 **Compute the expected daily alert volume** for three rules before enabling them.
- 7 **Deliberately break a log field name** and confirm your detection silently stops matching.
- 8 **Measure the false positive rate per rule** over a month, and identify rules to retire.
- 9 **Check your log retention period** and compare against typical intrusion discovery times.
- 10 **Confirm your logs survive compromise of the host** that emits them.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why design logging backwards, and what are the six event categories?
- 2 Name the five required fields and say which is most often missing.
- 3 Why must the 'who' be the real principal?
- 4 Name five high-signal detections, and say why low-volume ones are best.
- 5 Why is an untested detection a hope, and what breaks rules silently?
- 6 Give the base rate arithmetic and the predictable failure mode.
- 7 How do you raise the base rate rather than the accuracy?
- 8 Give four ways to manage volume, and the one nobody does.
- 9 Name four reasons detection programmes fail.
- 10 Which two controls decide whether investigation is possible?